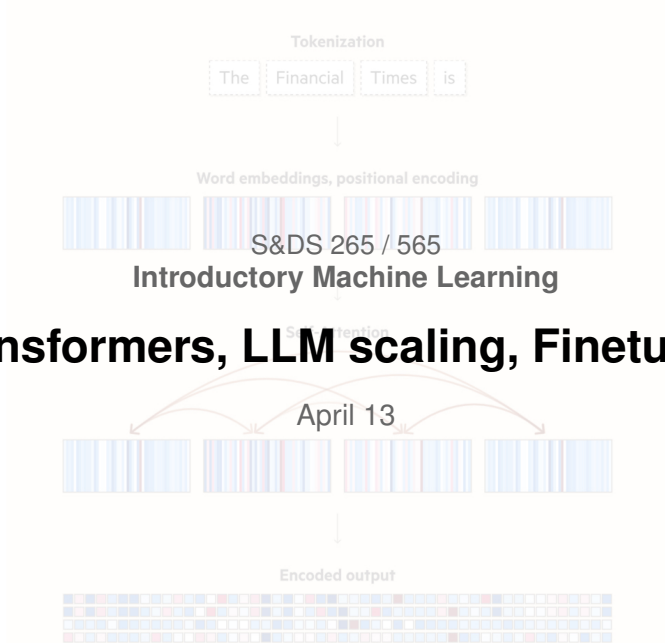




# Attention, please!

- Assignment 5 (last!) is out
- Quiz 5 (last!) this Wednesday
  - ▶ Q-learning, RL, autoencoders, LLMs
- Final exam, May 4, at 2pm
- Review sessions:
  - ▶ Yihan: Wed. April 29, 6pm
  - ▶ Dingyao: Thu. April 30, 8pm
  - ▶ Awni: Friday, May 1, 3pm

# For today

- Scaling laws
- Finetuning

**Recall: The Transformer architecture**

**Encoders and Decoders**

# Attention

- Attention is a type of alignment between related words
- Attention allows dynamic construction of word contexts, useful for predicting the next word
- Attention is computed using inner products, after mapping query and key to a common space
- Transformers process all words together, using layers of encoders and decoders, each with an attention component

# Demo

## Attention



This demo illustrates some of the concepts behind attention in transformers, using word embeddings. The goal is simply to show how the basic calculations underlying attention work, and how weighting embeddings with attention can change the words that are similar. As implemented in Transformers, attention can be notoriously difficult to understand.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
import gensim
```

# Implementation of Attention

Embedding for current position gets transformed based on similarity to embeddings at previous positions

$$\begin{aligned}e_t &\leftarrow e_t + \sum_{s < t} (\alpha_{st} W_V e_s) \\&= e_t + W_V \left( \sum_{s < t} \alpha_{st} e_s \right)\end{aligned}$$

where  $W_V \in \mathbb{R}^{d \times d}$  is a matrix.

# Implementation of Attention

Embedding for current position gets transformed based on similarity to embeddings at previous positions

The *attention weights* are computed in terms of similarity scores:

$$w_{s,t} = (W_K e_s)^T (W_Q e_t)$$

$$\alpha_{s,t} = \frac{\exp(w_{s,t})}{\sum_{s' < t} \exp(w_{s',t})} \equiv \text{Softmax}(w_{s,t})$$

where  $W_K \in \mathbb{R}^{d \times d}$  and  $W_Q \in \mathbb{R}^{d \times d}$  are two matrices.



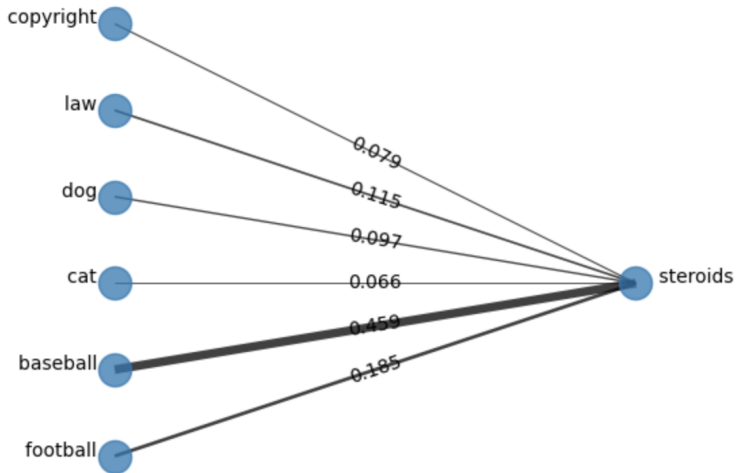
# Implementation of Attention

Embedding for current position gets transformed based on similarity to embeddings at previous positions

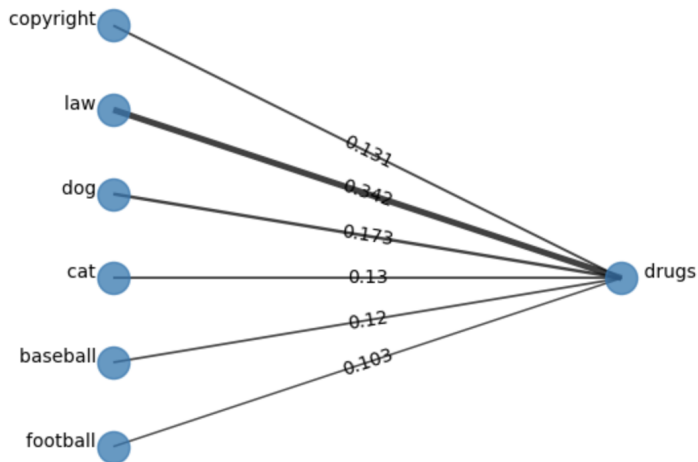
The *attention weights*  $\alpha_{st}$  sum to one:

$$\sum_{s < t} \alpha_{st} = 1$$

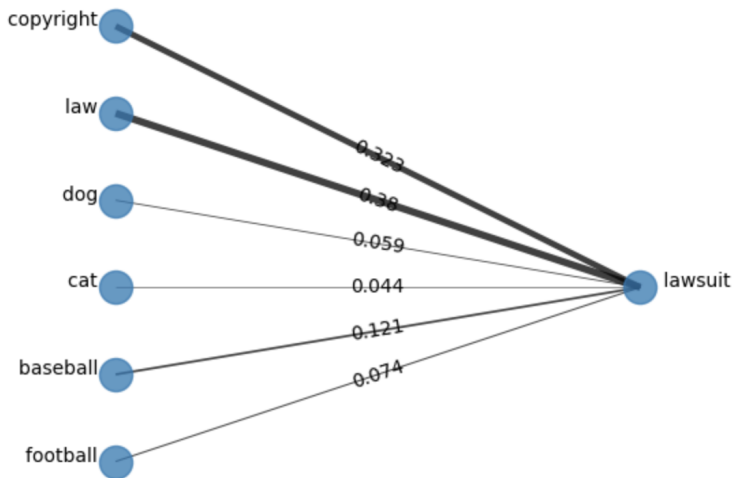
# Examples from demo



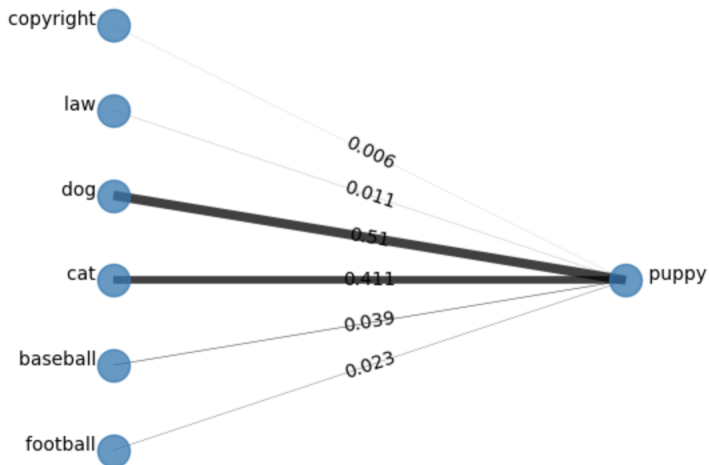
# Examples from demo



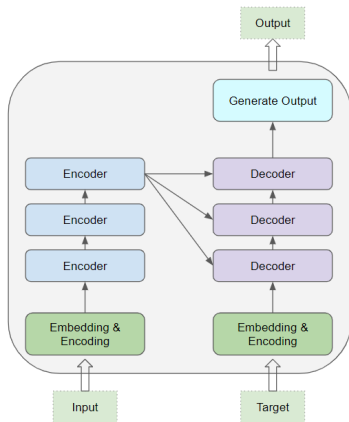
# Examples from demo



# Examples from demo



# Transformer architecture



# Transformer architecture

- Each Encoder transforms its input to generate a vector  $E$  for each source word
- Each Decoder transforms its input to generate a vector  $D$  for each target word
- The Decoder model uses *cross-attention* to attend to the Encoder vectors

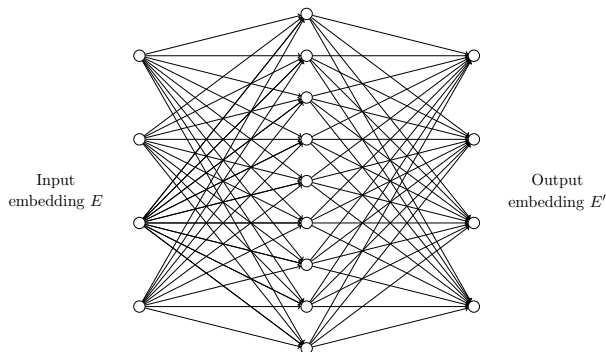
Encoder self-attention:  $Q \leftarrow E, K \leftarrow E, V \leftarrow E$

Decoder self-attention:  $Q \leftarrow D, K \leftarrow D, V \leftarrow D$

Decoder cross-attention:  $Q \leftarrow D, K \leftarrow E, V \leftarrow E$

# Adding nonlinearities

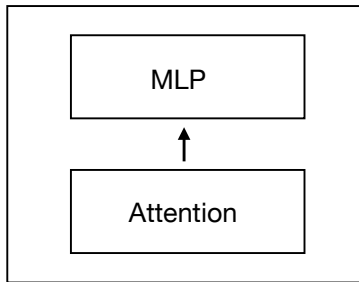
After attention, the mixed embeddings are transformed to vectors using an MLP:



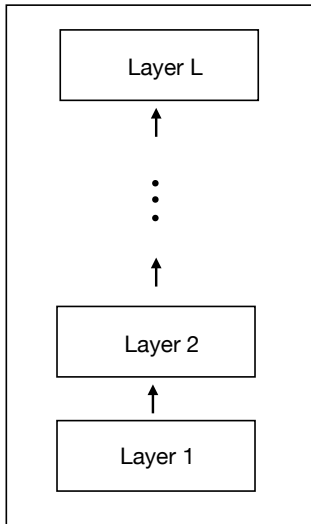
Creates new “features” to be used as embeddings in the next layer



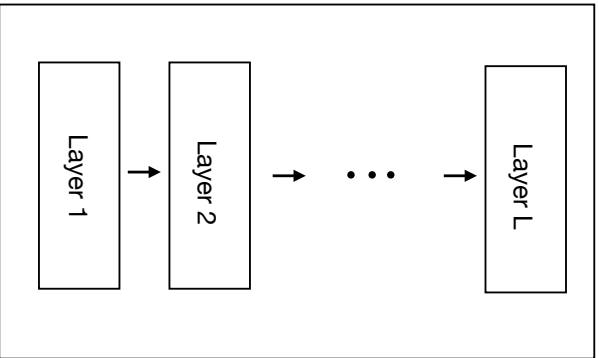
## Transformer Layer

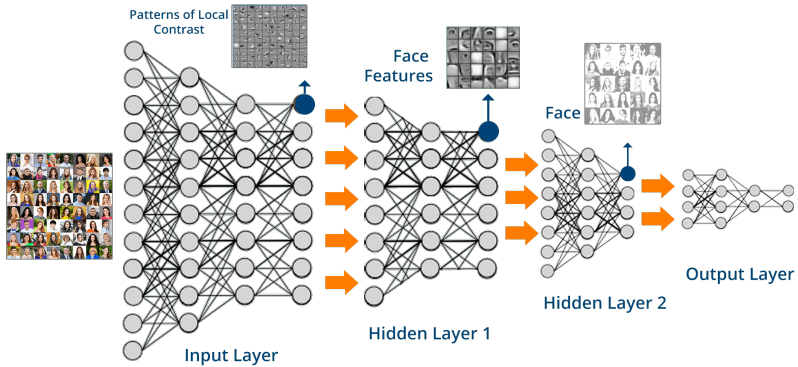


# Transformer



## Transformer





**Recall: The Transformer architecture**  
**How large are the models?**

# How does size of LLM scale?

Design choices: number of tokens  $V$ , dimension  $d$  of embeddings, and number of Transformer layers  $L$

- Embeddings have  $d \cdot V$  parameters
  - ▶  $d$  numbers for each token,  $V$  tokens

# How does size of LLM scale?

Design choices: number of tokens  $V$ , dimension  $d$  of embeddings, and number of Transformer layers  $L$

- Self-Attention has  $\approx 4d^2$  parameters
  - ▶  $d \times d$  matrix for queries (across heads)
  - ▶  $d \times d$  matrix for keys (across heads)
  - ▶  $d \times d$  matrix for values (across heads)
  - ▶  $d \times d$  matrix for combining heads

# How does size of LLM scale?

Design choices: number of tokens  $V$ , dimension  $d$  of embeddings, and number of Transformer layers  $L$

- MLP applied after has  $\approx 8d^2$  parameters
  - ▶ Two layers
  - ▶ Hidden layer has  $4 \cdot d$  neurons
  - ▶ Output layer has  $d$  neurons
  - ▶  $4d^2$  weights in each layer;  $8d^2$  weights overall



# How does size of LLM scale?

Design choices: number of tokens  $V$ , dimension  $d$  of embeddings, and number of Transformer layers  $L$

- Embeddings have  $d \cdot V$  parameters
- In each Transformer layer:
  - ▶ Attention has about  $4d^2$  parameters
  - ▶ MLP has about  $8d^2$  parameters
- Total trainable parameters:  $12d^2 \cdot L + d \cdot V$

# Size of models

## GPT-2 (2019)

$$d = 1,600$$

$$V = 50,257$$

$$L = 48$$

$$12 \cdot d^2 \cdot L + d \cdot V \approx 1.5 \text{ billion parameters}$$

- ▶ Each GPT-2 Transformer layer has about 30 million parameters

# Size of models

GPT-3 training data<sup>[1]:9</sup>

| Dataset                      | # tokens    | Proportion within training |
|------------------------------|-------------|----------------------------|
| <a href="#">Common Crawl</a> | 410 billion | 60%                        |
| WebText2                     | 19 billion  | 22%                        |
| Books1                       | 12 billion  | 8%                         |
| Books2                       | 55 billion  | 8%                         |
| Wikipedia                    | 3 billion   | 3%                         |

# Size of models

## GPT-3 (2020)

$$d = 12,288$$

$$V = 50,257$$

$$L = 96$$

$$12 \cdot d^2 \cdot L + d \cdot V \approx 173 \text{ billion parameters}$$

- ▶ Each GPT-3 Transformer layer has about 1.8 billion parameters



Original painting by Johannes Vermeer; transformed (pixelated) by acagastya\_, CC0, via Wikimedia Commons

ANNALS OF ARTIFICIAL INTELLIGENCE

## CHATGPT IS A BLURRY JPEG OF THE WEB

*OpenAI's chatbot offers paraphrases, whereas Google offers quotes. Which do we prefer?*

By Ted Chiang

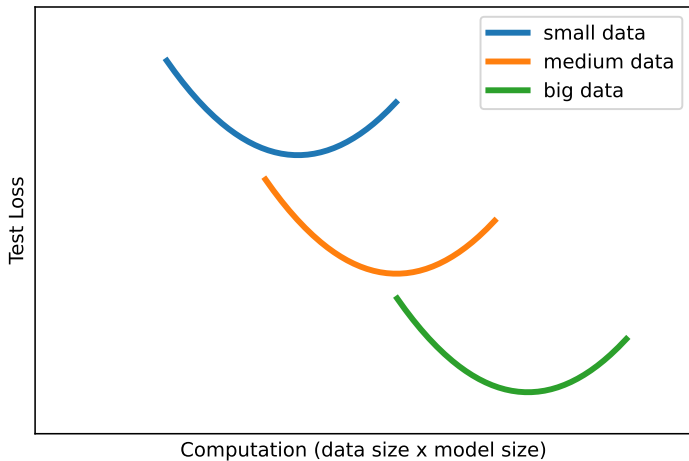
February 9, 2023

# Back of the envelope calculation

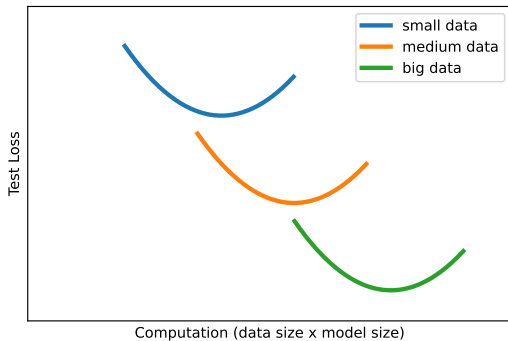


- LLM loss on test data is  $\approx 1.9$  nats/token
- Translates to entropy of about  $1.9 / \log_e(2) \approx 2.74$  bits/token
- So, *lossless* coding of 500 billion tokens takes  $\approx 171$  gigabytes
- GPT-3's 175 billion params uses 346 gigabytes (16-bit precision)
- GPT-3 could, in principle, memorize most of the internet!

# Scaling behavior of LLMs: Performance

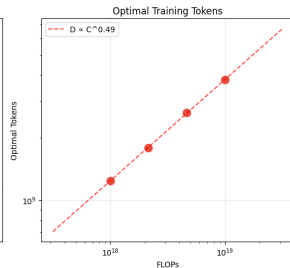
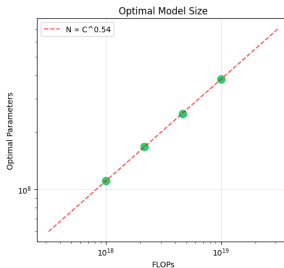
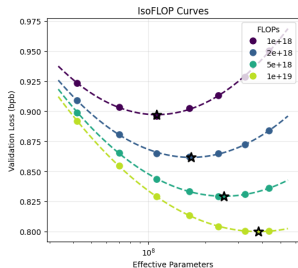


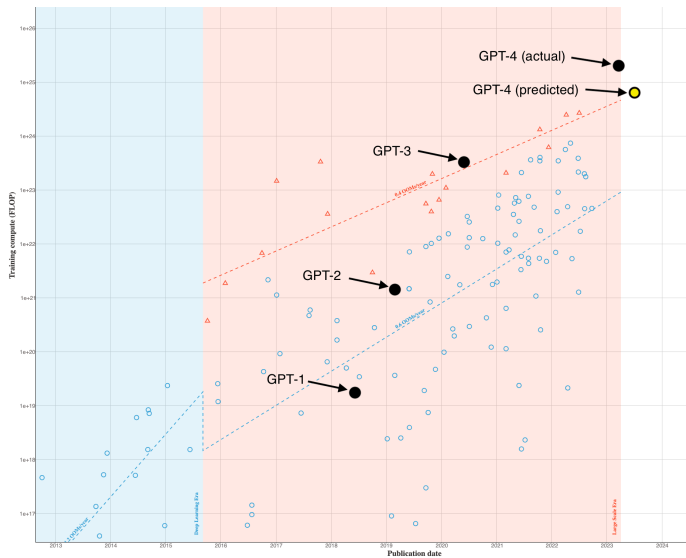
# Scaling behavior of LLMs: Performance

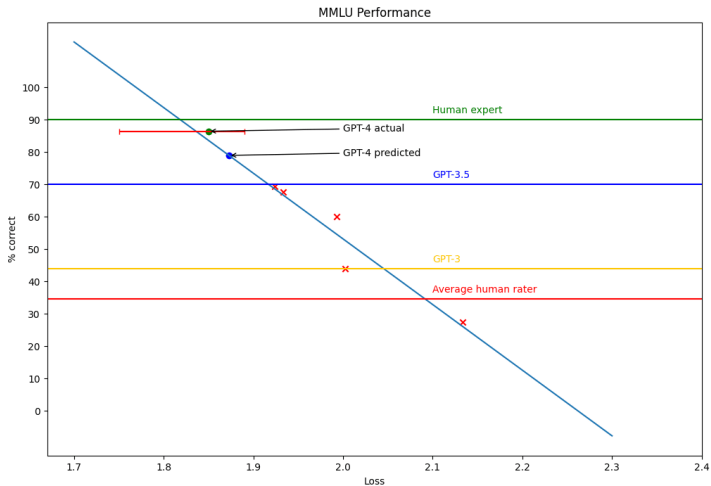




# Scaling behavior of LLMs: Performance







MMLU: Massive Multitask Language Understanding, <https://en.wikipedia.org/wiki/MMLU>,  
<https://paperswithcode.com/dataset/mmlu>

## *Sutton's "Bitter Lesson" (2019)*



“The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin.”

**Generating text**

# **The generation process**

# Rolling a D50,257



- ▶ Transformer assigns a score to each token
- ▶ Converted to a probability
- ▶ Generating next token equivalent to rolling weighted 50,257-sided die



# Generation

Generation algorithm has two “knobs” to control:

- 1 *Top-k*: Restricts to just the top  $k$  tokens
  - ▶ Low  $k$ : Low variability
  - ▶ High  $k$ : Higher variability



# Generation

Generation algorithm has two “knobs” to control:

- ② *Temperature  $T$* : Lower temperature favors more likely tokens
  - ▶ Low temperature: Low variability
  - ▶ High temperature: High variability

# Generation

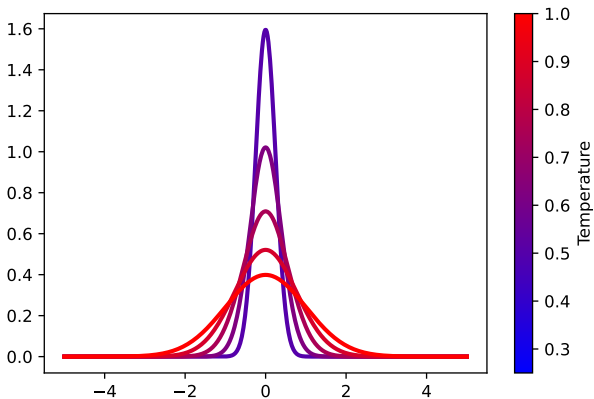
At temperature  $T$ , if the scores of the top  $k$  tokens are

$$\text{logit}_1, \text{logit}_2, \dots, \text{logit}_k$$

the weights on the die are proportional to

$$e^{\text{logit}_1/T}, e^{\text{logit}_2/T}, \dots, e^{\text{logit}_k/T}$$

# Temperature



# Temperature $T = 1$

One day, Timmy met a monster. The monster was big and green, but it was harmless. Timmy was scared, but he wanted to be brave. He walked closer and closer to the monster. Suddenly, the monster snapped its fingers! Timmy was so scared that he started to cry. The monster said, "Don't be scared! I'm harmless!" But Timmy was still sc

One day, Timmy met a monster. The monster had big teeth and green skin. Timmy was scared and ran away. But the monster called out, "Wait! I don't want to hurt you. I just want to be your friend." Timmy stopped and looked back. "Really?" he asked. "Yes, I permit you to be my friend," the monster replied. Timmy was surprised. He didn't expect

# Temperature $T = 0$

One day, Timmy met a monster. The monster was big and scary. Timmy was scared and ran away. The monster chased Timmy. Timmy ran and ran until he found a big tree. He climbed up the tree and the monster couldn't reach him. Timmy was safe. Timmy saw a bird and said, "Hello bird, can you help me? The monster is chasing me." The bird said, "

One day, Timmy met a monster. The monster was big and scary. Timmy was scared and ran away. The monster chased Timmy. Timmy ran and ran until he found a big tree. He climbed up the tree and the monster couldn't reach him. Timmy was safe. Timmy saw a bird and said, "Hello bird, can you help me? The monster is chasing me." The bird said, "

# Temperature $T = 1.5$

One day, Timmy met a monster. The monster was big and scary. Timmy didn't know what to do. He felt bitter because he wanted to play with the monster. The monster said, "Hello, Timmy. Can you show me around the forest?" Timmy wasn't sure if he believed him or not. He said, "I'm sorry, do you have room for crawlers?" The monster guesses sad

One day, Timmy met a monster. The monster was big and scary, but Timmy wasn't scared. "Hi," said Timmy. The monster was surprised. He couldn't understand what Benny had said. "What are you up to?" he asked. Timmy picked him up one by one and he saw doziles of Ell Party coming in on a streak street. "I amdri catientased!"

Complete this sentence: The quick brown fox

"The quick brown fox jumps over the lazy dog."

(A classic — it's a pangram containing every letter of the alphabet.)

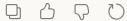


Now complete it at a slightly higher temperature

"The quick brown fox sprinted through the amber dusk, chasing something it would never quite catch."

Again

"The quick brown fox, inexplicably wearing a monocle, demanded to speak with the manager of the forest."





Now complete it at temperature  $T=2.0$

"The quick brown fox xylophone underneath several Tuesday moons, dripping with forgotten algebra."

(Illustrating the point — high temperature completions become incoherent as the probability distribution over tokens flattens out and unlikely/random tokens get sampled more frequently.)

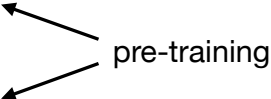
**Making the models useable**

# **Finetuning**

# Recall: Machine learning frameworks

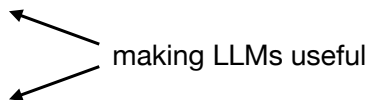
- Supervised
- Unsupervised, self-supervised
- Reinforcement learning
- Representation learning

# Recall: Machine learning frameworks

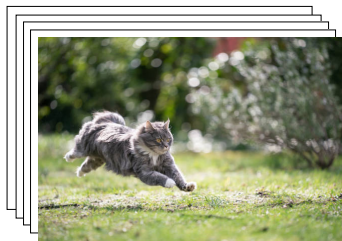
- Supervised
  - Unsupervised, self-supervised
  - Reinforcement learning
  - Representation learning
- 
- pre-training
- The diagram shows two arrows originating from the text 'pre-training' on the right. One arrow points diagonally up and to the left towards the text 'Unsupervised, self-supervised'. The other arrow points diagonally down and to the left towards the text 'Representation learning'.

# Recall: Machine learning frameworks

- Supervised
- Unsupervised, self-supervised
- Reinforcement learning
- Representation learning



# Recall: Supervised learning



Goal: Accurately assign label  $y \in \{\text{dog}, \text{cat}\}$  to an image  $x$ .

“Learn” from a large database of examples  $\{(x_i, y_i)\}$ .

# Recall: Supervised learning

Fugetsu-Do, a mochi store in the Little Tokyo neighborhood of Los Angeles, has existed for 121 years, with a lasting impact on its community.

ロサンゼルスのリトル東京地区にある餅屋「風月堂」は 121 年間存在し、地域社会に永続的な影響を与えています。

Goal: Accurately translate input  $x_1, \dots, x_m$  to output  $y_1, \dots, y_m$ .

“Learn” from a large database of examples  $\{(x, y)\}$ .

This is called *sequence-to-sequence learning* (seq2seq)

# Finetuning

Finetuning takes a pre-trained LLM, and feeds it a database of examples of how it should “translate” input to output.

Examples are here:

`huggingface.co/datasets/HuggingFaceH4/no\_robots`

Categories: Generation, Open QA, Brainstorm, Chat, Rewrite, Summarize, Coding, Classify, Closed QA, Extract



# Example training data

Datasets: HuggingFaceH4 / no\_robots like 539 Follow Hugging Face H4 1.46k Dataset card Data Studio

Split (2)  
train · 9.5k rows

Search this dataset

| prompt<br>string · lengths                                                                                                                                           | prompt_id<br>string · lengths                                    | messages<br>list · lengths                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | category<br>string · classes |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| <br>0-955 89.8%                                                                                                                                                      | <br>64 100%                                                      | <br>2-4 91.6%                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | <br>Brainstorm 11.2%         |
| Coca-Cola from the point of...                                                                                                                                       | 3c069cf2da2e71                                                   | about buying Coca-Cola from...                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                              |
| I love reading, but I have a hard time retaining what I...                                                                                                           | 18131b42b0d497945f220e162d291741b64750428b7bd0136e602d8822d08d1  | [ { "content": "I love reading, but I have a hard..." }                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Generation                   |
| Write a 300-word blog post about taking care of a guinea...                                                                                                          | b76668bb05d4e2a83f6f4b4a395e3a002c10b42c7f93ae1b2a51a0a718422cbf | [ { "content": "Write a 300-word blog post about taking..." }                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Generation                   |
| I'm looking to visit Berlin but have no clue what neighborhoods to stay in. I'll be on a bit of a budget while there, what are some good neighborhoods to check out? | 5456d16df6d666f7bf3de5529d3f2577d797d4b1f45d135f75458See9177726d | [ { "content": "I'm looking to visit Berlin but have no clue what neighborhoods to stay in. I'll be on a bit of a budget while there, what are some good neighborhoods to check out?", "role": "user" }, { "content": "- Charlottenburg-Wilmersdorf: This is the best area for a family to stay. It used to be its own town, separate from the city. There are a ton of historical sites here, like the Charlottenburg Palace, and it's a very safe part of the city.\n\n- Kreuzberg: Kreuzberg is the best place to go if you want nightlife. There are" } | Brainstorm                   |

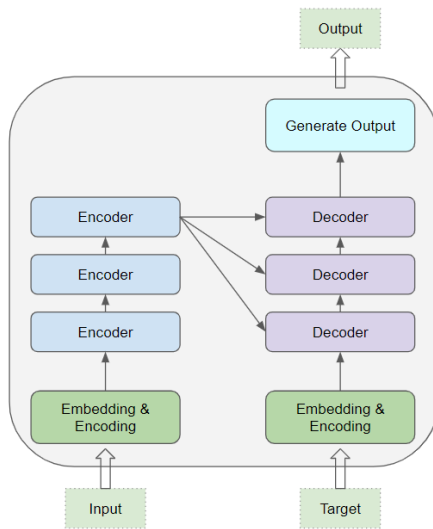
< Previous 1 2 3 ... 95 Next >

# Training process

- ① Prompt is read in and embedding / encoding vectors are processed just as in generation.
- ② Target sequence is generated (predicted) token-by-token
- ③ Parameters are iteratively updated to make the target sequences more and more likely

*After training is complete, updated LLM has been “finetuned” to the tasks in the training data*

# Transformer finetuning architecture



# Finetuning fineprint

- The LLM is huge (175 billion parameters for GPT-3)
- Finetuning training data are too small to adjust all the parameters
- Special techniques are used to make “small” changes to the model (e.g. LoRa)

**Making the models useful**

# **Reinforcement learning**

# Reinforcement learning

- An agent interacts with an environment
- The actions the agent takes change the state of the environment
- The agent receives rewards for each action, and seeks to maximize the total cumulative reward

*Reinforcement learning is a framework for sequential decision making to achieve a long-term goal.*

# Reinforcement learning: Motivating examples

- Learning to walk or ride a bike
- A robot vacuum cleaning up the house
- Playing chess, backgammon, Atari games, etc.
- Drug discovery, personalized health, energy management

# Characteristics

- RL is inherently sequential
- In between supervised and unsupervised learning
- Agent can't act too greedily; needs to be strategic

The aim of RL is to learn to make optimal decisions from experience



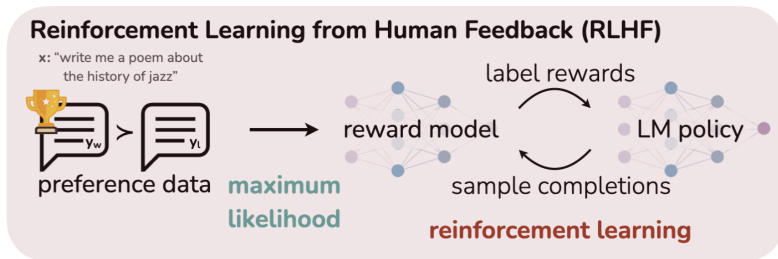
# RLHF

## RLHF — Reinforcement Learning from Human Feedback

- Key to RL is the *reward signal*
- First step of RLHF: Learn reward function from human preferences
- Given reward, learn policy to improve LLM outputs

# RLHF

## RLHF — Reinforcement Learning from Human Feedback



# Policy optimization

In Reinforcement Learning, a *policy* is way of choosing actions to optimize long-term rewards

- Here, the policy governs which tokens to generate
- The rewards are the desirable characteristics of text as learned from human judges
- The policy is iteratively improved to generate text that is
  - ▶ *Helpful, Correct, Coherent, Appropriately Simple/Complex*
- These characteristics change with the domain

# Summary

- Transformer architecture
- Scaling laws: Diminishing returns?
- Generating new text
- Making the models useful:
  - ▶ Finetuning: Training on human-generated responses
  - ▶ RLHF: Learn to predict human judgements of quality (reward)
  - ▶ RLHF: Use estimated rewards to improve quality (policy)